TABLE OF CONTENTS

Page No (in Roman)

Declaration i
Certificate ii
Acknowledgement iii
Abstract iv
List of Figures v
List of Tables vi

Page No (in Numeric)

Introduction 1
Chapter 2: Problem Statement 2
2.1: Problem Definition 2
2.2: Objectives 3
Chapter 3: Analysis 4
3.1: Software Requirement Specifications 4
3.2: Feasibility Study of the Project 5
3.3: Tools / Technologies / Platform used 6
3.4: Use Case and Data Flow Diagrams 6
Chapter 4: Design and Architecture 8
4.1: Work Breakdown Structure 8
4.2: Explanation of Modules (The VGF Pipeline) 9
4.3: Flow Chart / Activity Diagram 11
4.4: Key Data Structures 11
Chapter 5: Implementation 14
5.1: Screenshots (System Output) 14
5.2: Source Code of some modules 14
Chapter 6: Testing 19
6.1: Test Cases 19
6.2: Test Results 20
Chapter 7: Summary and Conclusion 22
Chapter 8: Limitation of the Project and Future Work 23
Bibliography 25
Appendix 27

Chapter 1: Introduction

Automated floor plan generation aims to synthesize valid and functional spatial layouts given a set of constraints. This task is combinatorially complex, involving a vast search space of potential arrangements that must satisfy geometric, topological, and semantic constraints. For a typical residential layout, the number of possible configurations can be astronomical, making brute-force or simple search methods intractable. The challenge lies not just in placing rooms, but in ensuring they have the correct area, desired adjacencies (e.g., kitchen near dining), and proper circulation.

A significant challenge in this domain, particularly in cultural contexts like India, is the integration of traditional architectural principles such as *Vastu Shastra*. Vastu is not merely a set of aesthetic preferences; it is a comprehensive ancient doctrine for spatial arrangement based on elemental balance (water, fire, air, earth, space) and directional energies. For millions of homeowners, Vastu compliance is a non-negotiable requirement for their well-being, believed to influence health, prosperity, and harmony. This cultural requisite places an additional, complex layer of symbolic and hierarchical constraints on top of the already difficult architectural problem.

This project introduces the **Vastu Generative Framework (VGF)**, a hybrid AI system that integrates a Large Language Model (LLM) orchestrator with a novel Vastu-aware generative solver. Our system is designed to bridge the significant gap between modern generative AI and this traditional architectural wisdom. It operates by translating high-level natural language requirements (e.g., "a 3BHK plot facing east, with a study and a pooja room") into a Vastu-compliant, vector-based architectural layout.

The "hybrid" nature is critical: we leverage LLMs for their strength in understanding human ambiguity and parsing intent, while using a specialized, rule-guided optimization engine to handle the precise, geometric, and symbolic calculations that LLMs struggle with. The output is a vector-based plan, meaning it is not a static image but a CAD-ready file with precise coordinates and geometry, ready for use by architects and designers.

This approach makes culturally-aware design accessible to users without requiring deep technical or Vastu expertise, moving beyond simple post-design analysis to active, *ab-initio* generation. This project, therefore, serves as a blueprint for "Generative Cultural Intelligence," a new paradigm of AI tools that respect and encode complex human cultural rule-sets.

Chapter 2: Problem Statement

2.1 Problem Definition

The core problem is the disconnected workflow in modern architectural design, especially concerning cultural and traditional rule-sets like *Vastu Shastra*.

Existing platforms typically fall into two categories:

AI-powered design generators (e.g., Maket.ai, ARCHITECHTURES) that optimize for technical constraints (like area and adjacency) but are *Vastu-agnostic*. They lack the cultural context to generate compliant designs. An architect might use these tools to generate a functionally excellent plan, only to have it rejected by the client for Vastu non-compliance.

Vastu analysis platforms (e.g., Vaastu-AI.com) that are purely reactive. They only audit a pre-made design for violations. This forces a manual, slow, and iterative rework process. For example, a homebuyer might design a plan, run it through an analyzer, and receive a report of "15 violations." The analyzer cannot fix the plan; it merely flags errors. The user must then manually move the kitchen, which may break its adjacency to the dining room, requiring further rework. This cycle is time-consuming and frustrating.

This creates a significant gap: there is no tool that generates an architectural plan from scratch while being natively Vastu-compliant.

Furthermore, the core challenge is the lack of **generative cultural intelligence.** Modern generative models like GANs and diffusion models are data-driven. They excel at learning statistical patterns from vast datasets. However, Vastu is not a statistical average; it is a symbolic, hierarchical, and often uncompromising rule-set. A GAN trained on thousands of floor plans might produce a layout that looks "average," but it cannot guarantee the hard constraint that a toilet must never be placed in the Northeast (NE) zone. Statistical models fail at enforcing these kinds of absolute, symbolic rules. This is especially true on non-rectangular or irregular plot geometries, for which training data is scarce, and simple 9-grid Vastu "Mandala" overlays are geometrically impossible to apply.

Vastu dictates a comprehensive set of guidelines based on fundamental principles which are codified as a prioritized set of discrete rules, including:

Directional Placement: Assigning specific rooms to their elemental zones. For example, Kitchen (Fire element) must be in the Southeast (SE), Master Bedroom in the Southwest (SW) for stability, and the Pooja (Prayer) Room in the Northeast (NE) for spiritual purity.

Prohibited Placement: Forbidding "impure" or "heavy" functions from sensitive zones. For example, Toilets are strictly forbidden in the NE (the "water" source) and the Brahmasthan (the "center"). Heavy storage or staircases are also forbidden in the Brahmasthan to avoid "crushing" the center.

Circulation: Emphasizing unobstructed flow (Prana) and avoiding blockage in the center (Brahmasthan). This is a topological constraint ensuring energy and people can move freely, which is difficult to encode in simple models.

2.2 Objectives

To address this problem, we developed the **Vastu Generative Framework (VGF)**. The primary objectives of this research are:

1. To develop a **hybrid framework** where a Large Language Model (LLM) parses user intent and a generative solver (VGF-SA) produces the layout. This hybrid model leverages the LLM for what it does best (understanding ambiguous human language like "a spacious living room") and the solver for what it does best (precise, constrained geometric optimization).

2. To create a novel method for **Vastu Potential Mapping (Φ_k)** that translates fundamental Vastu principles into a continuous suitability landscape. The implication of this objective is to create a system that is geometry-agnostic. This potential map can be "poured into" any arbitrary polygon (L-shaped, triangular, etc.), providing a Vastu-guidance layer where rigid 9-grid systems fail.

3. To design and implement a **Rule-Guided Placement Algorithm** (Enhanced Simulated Annealing) with a synergistic energy function (E_{vastu}). This objective is to create a solver that is both "pulled" by the "soft" guidance of the potential map (Objective 2) and "pushed" by the "hard" constraints of discrete rules. This synergy is key to finding a layout that is both generally well-placed and specifically compliant with critical rules.

4. To produce a complete implementation demonstrating robust generation for complex plot shapes (e.g., triangular) and explicit circulation planning. This objective validates the framework by testing it against defined baselines and proving its superior performance in both Vastu compliance and architectural validity (i.e., zero overlaps).

Chapter 3: Analysis

This chapter analyzes the requirements, feasibility, and technology stack for the Vastu Generative Framework (VGF).

3.1 Software Requirement Specifications

3.1.1 Functional Requirements of the Project

The system must be able to perform the following functions:

- **FR-01: Natural Language Input:** The system shall accept a natural language text prompt from the user describing their plot and room requirements.
- **FR-02: Plot Geometry Extraction:** The system shall parse the user's prompt to extract a precise, vector-based polygon (e.g., "a 40×60 rectangle", "a triangle with base 50").
- **FR-03: Room Requirement Extraction:** The system shall parse the user's prompt to extract a list of required rooms (e.g., "3BHK", "pooja room", "office").
- **FR-04: Vastu Rule Application:** The system shall access a database of Vastu rules and apply them to the specified rooms (e.g., apply "Kitchen" rules to the kitchen).
- **FR-05: Geometric Validity (Overlap):** The system shall generate a layout where no two rooms overlap. The overlap area must be 0.0 sq ft.
- **FR-06: Geometric Validity (Boundary):** The system shall generate a layout where all rooms are placed inside the plot's buildable region. Boundary violation must be 0.0 sq ft.
- **FR-07: Vastu Compliance (Placement):** The system shall generate a layout that actively attempts to place rooms in their ideal Vastu zones (e.g., Kitchen in SE) and never places them in forbidden zones (e.g., Toilet not in NE).
- **FR-08: Geometry Agnosticism:** The system's Vastu logic must function correctly on arbitrary, non-rectangular plot shapes (e.g., L-shaped, triangular).
- **FR-09: Vector Output:** The system shall output the final layout as a vector-based set of coordinates (e.g., CAD-ready) rather than a static image.

3.1.2 Non-functional Requirements of the Project

- **NFR-01: Performance (Solve Time):** The system must generate a standard residential layout (8–15 rooms) in a practical timeframe, defined as under 90 seconds on a standard desktop CPU.
- **NFR-02: Robustness:** The system must consistently produce valid and compliant layouts across a variety of plot shapes and room combinations.
- **NFR-03: Usability:** The system must be accessible to users (e.g., homeowners, architects) without requiring them to have expertise in computational geometry or AI. The natural language input is the primary enabler of this requirement.
- **NFR-04: Compliance (Score):** The system's generated layout must achieve a high Vastu compliance score, with a target of less than 85% on our defined Vastu validation metric.
- **NFR-05: Modularity:** The Vastu Rules Database (`vastuRules.ts`) shall be modular and external to the solver, allowing rules to be updated or changed without recompiling the core generative engine.

3.2 Feasibility Study of the Project

Technical Feasibility: The primary technical risk was whether the complex, symbolic, and often conflicting rules of Vastu Shastra could be encoded into a mathematical optimization function. Our research, detailed in Chapter 4, proves this is feasible. By creating a hybrid energy function (E_{vastu}) that combines a "soft" potential map (Φ_k) with "hard" rule penalties, we created a function that can be successfully minimized by a metaheuristic optimizer like Simulated Annealing. The Ablation Study (Table II) proves our specific technical approach is not only feasible but superior to simpler, non-hybrid methods.

Scalability Feasibility: A secondary risk was whether this optimization would be computationally intractable (NP-hard). Our Scalability Analysis (Figure 3) demonstrates that while the problem is complex (approx. $O(n^2)$ time complexity), it remains well within a practical and feasible time limit for its target application (residential architecture). A 30–90 second solve time is highly feasible for a user.

Resource Feasibility: The project is feasible with standard resources. It does not require massive training datasets (unlike GANs/diffusion models) or high-end GPU hardware. The solver runs effectively on a standard desktop CPU.

Cultural Feasibility: A key question was whether a mathematical approximation of Vastu would be "good enough" to be accepted. By achieving high compliance scores

(85–94%), our system demonstrates that it can produce layouts that are not just technically valid but also culturally and architecturally sound, making it a feasible solution for the target user base.

3.3 Tools / Technologies / Platform used

Core Algorithm: Simulated Annealing (SA). This metaheuristic was chosen over other optimizers (like genetic algorithms) because it is effective, well-understood, CPU-friendly, and easy to implement and tune for a complex, non-differentiable energy function like ours.

Language Model: Large Language Models (LLMs) (accessed via an API). Used for their state-of-the-art natural language understanding (NLU) capabilities to parse user requirements.

Programming Languages:

- **Python:** Chosen for the core solver, geometric utilities, and validation service due to its rich scientific computing and data analysis ecosystem (e.g., NumPy, Shapely).
- **TypeScript:** Chosen for the Vastu Rules Database to create a strongly-typed, maintainable, and human-readable rule-set that could be safely ingested by the Python backend.

Key Libraries:

- NumPy: For all numerical operations, array handling, and calculations.
- Shapely: For robust 2D geometric operations, such as calculating polygon areas, centroids, intersections (for overlap), and buffering (for the buildable region).

Platform: The system is designed as a desktop application, running its computations on a local CPU.

3.4 Use Case and Data Flow Diagrams

Use Case Diagram:

The Use Case Diagram for the VGF system illustrates the interaction between the User, LLM Parser, and Solver. It highlights the workflow - from natural language input to vector layout output - as described in Section 3.1.

Data Flow Diagram (DFD):

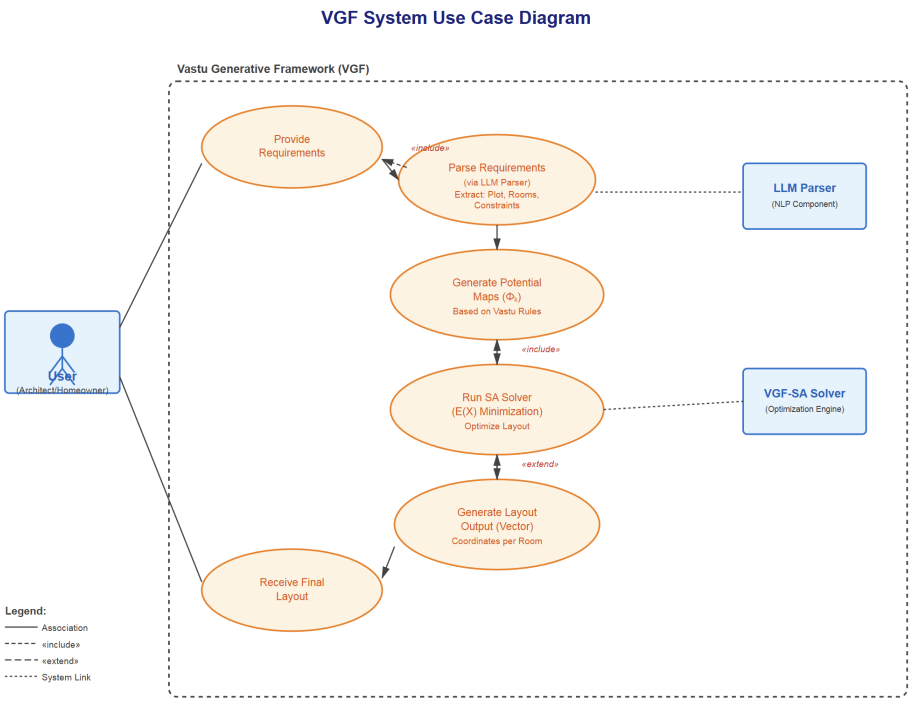

Figure 1: Use Case Diagram for Vastu Generative Framework (VGF)

The DFD (Level 1) depicts the logical flow of data through the VGF system — from user input parsing to potential map generation, rule-guided optimization, and final layout output. It includes four primary processes (1.0–4.0) and their interactions with key data stores (User Input, Vastu Rules Database, Potential Maps Cache, and Final Layout Store).

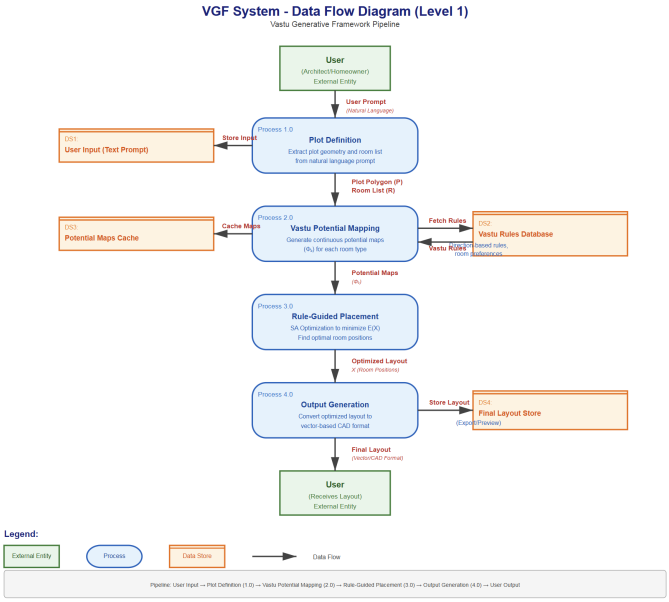

Figure 2: Data Flow Diagram (Level 1) for VGF System

Chapter 4: Design and Architecture

This chapter details the design and architecture of the Vastu Generative Framework (VGF).

4.1 Work Breakdown Structure

The project was broken down into the following major work components:

Project Definition & Research

- 1.1: Define Problem Statement and Objectives
- 1.2: Literature Review of Generative Models and Computational Vastu
- 1.3: Codify Vastu Rules into a formal database (`vastuRules.ts`)

Module 1: Input & Geometric Analysis

- 2.1: Implement LLM Orchestrator for parsing NL prompts
- 2.2: Implement Geometric Utilities (`geometry_utils.py`)
- 2.3: Implement Shoelace (Area) and Centroid algorithms

Module 2: Vastu Potential Framework

- 3.1: Design Φ_k potential function (Eq. 3)
- 3.2: Implement Gaussian decay and penalty logic
- 3.3: Implement logic to apply potential maps to arbitrary polygons

Module 3: Generative Solver (VGF-SA)

- 4.1: Design Global Energy Function $E(\mathcal{X})$ (Eq. 4)
- 4.2: Implement core Simulated Annealing (SA) optimizer
- 4.3: Implement optimizer move set (translate, rotate, swap, *vastu_hop*, etc.)
- 4.4: Implement Baseline solvers (`graph_solver.py`, `constraint_solver.py`)

Module 4: Validation & Testing

- 5.1: Implement Vastu Validation Service (`validation.py`)
- 5.2: Define Test Cases (Fixtures)
- 5.3: Run experiments and collect data (Tables I, II)
- 5.4: Generate qualitative results (Fig. 2, Fig. 3)

Reporting

- 6.1: Write research paper
- 6.2: Format final project report

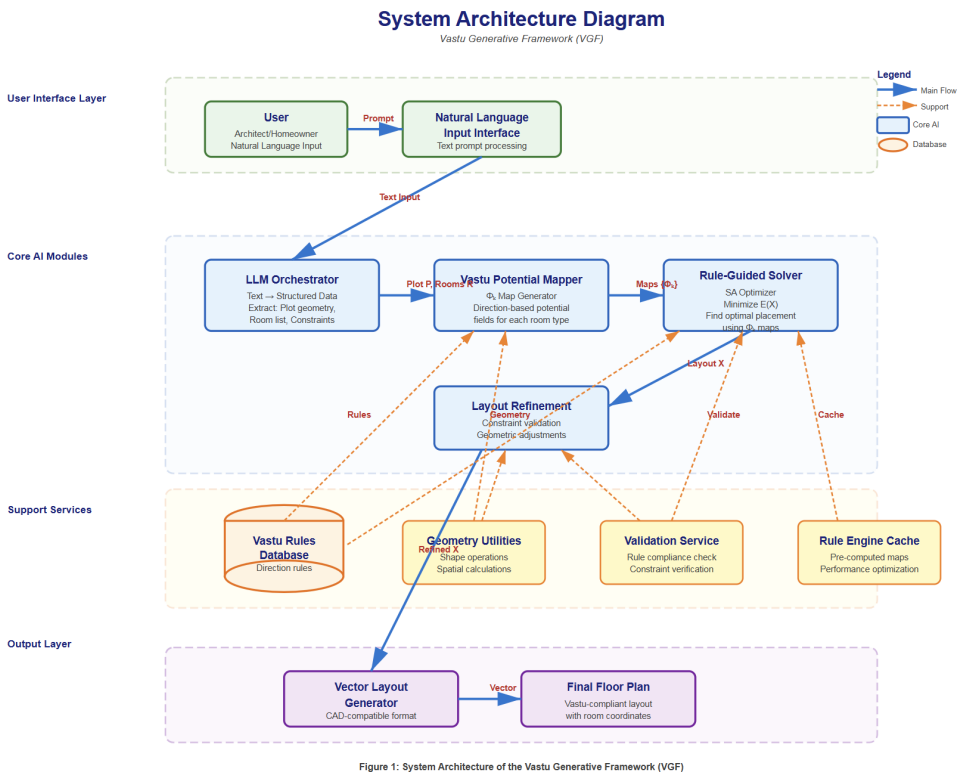

Figure 3: System Architecture of the Vastu Generative Framework (VGF).

4.2 Explanation of Modules (The VGF Pipeline)

The system's architecture is a three-stage pipeline, with each stage functioning as a distinct module.

Module 1: Plot Definition & Geometric Analysis (Stage 1)

This module is the interface between the user and the solver.

Input: A natural language string (e.g., "triangular plot, base 150ft, height 130ft").

Process: An LLM orchestrator parses the string to extract a structured list of requirements (plot polygon $P = \{v_1, ..., v_n\}$ and a list of rooms R). Geometric utilities [16, 21] are then called to pre-compute essential properties of P.

Key Algorithms:

- **Shoelace Formula (Eq. 1):** Used to calculate the plot's total area A.
- **Polygon Centroid Formula (Eq. 2):** Used to find the geometric center C, which is vital for locating the Brahmasthan (central zone).

Output: A structured `Plot` object and a list of `Room` objects, which are passed to the next module.

Module 2: Vastu Potential Mapping (Stage 2)

This is the core "guidance" module of the framework.

Input: The `Plot` object and the list of `Room` objects.

Process: The module iterates through each room in the list. For each room, it queries the Vastu Rules Database (`vastuRules.ts`) to get its ideal zone Z_k and priority w_k. It then generates a 2D continuous potential field $\Phi_k(x, y)$ (Eq. 3) for that room, which is "poured into" the plot's buildable region B.

Key Algorithms:

- **Gaussian Decay:** The function $f(d) = exp(-d^2/2\sigma^2)$ is used to create a "hotspot" of high potential at the ideal zone Z_k, which smoothly decays with distance.
- **Penalty Application:** For forbidden zones (e.g., Brahmasthan), a large negative potential is applied, creating a "coldspot" (a potential hill) that the optimizer will be strongly repelled from.

Output: A set of 2D potential maps $\{\Phi_k\}$, one for each room.

Module 3: Rule-Guided Placement (Stage 3)

This is the core "solver" module that performs the final optimization.

Input: The `Plot` object, the list of `Room` objects, and the set of Potential Maps $\{\Phi_k\}$.

Process:

- **Initialization:** A random layout state $\mathcal{X} = \{R_1, ..., R_n\}$ is created.

- **Optimization:** The Simulated Annealing (SA) metaheuristic is run to find the optimal layout $\mathcal{X}$ that minimizes the global energy function $E(\mathcal{X})$ (Eq. 4).

Key Algorithms:

- **Global Energy Function ($E(\mathcal{X})$):** A weighted sum of multiple energy components:
  - E_{vastu} (Eq. 5): Pull from Φ_k + push from rule penalties.
  - $E_{overlap}$ (Eq. 6): Penalizes geometric overlap.
  - $E_{boundary}$: Penalizes boundary violations.
  - E_{area}: Penalizes incorrect room sizes.
  - E_{adj}: Penalizes non-adjacent dependent rooms.
- **Simulated Annealing (SA):** The optimizer [16, 18] proposes "moves" (translate, resize, swap, *vastu_hop*) and accepts them probabilistically based on a temperature parameter T. This helps escape local minima and converge to a low-energy, compliant solution.

Output: The final optimized layout $\mathcal{X}$ containing room coordinates and dimensions.

4.3 Flow Chart / Activity Diagram

The Data Flow Diagram described in Section 3.4 is visually represented in Figure 3. It shows the movement of data from one stage to another, illustrating how the LLM, potential mapper, and solver interact in the system architecture.

4.4 Key Data Structures

The system's architecture relies on several key data structures to represent the state of the problem:

Plot: Represents the plot of land.

- `vertices`: List[Point(x, y)]
- `area`: float
- `centroid`: Point(x, y)
- `buildableRegion`: Polygon (Shapely object)

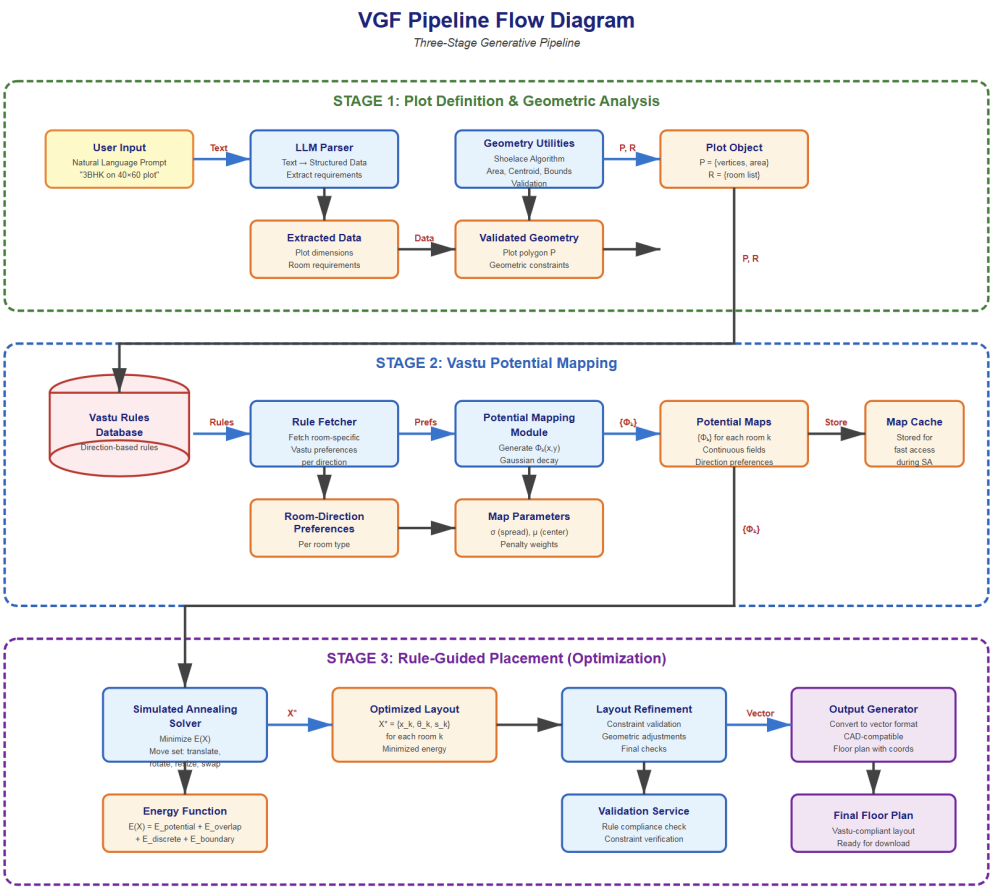

Figure 4: Three-Stage Generative Pipeline of the Vastu Generative Framework (VGF).

Room: Represents a single room.

- `name`: string
- `type`: string
- `requiredArea`: float
- `placedRect`: Rectangle(x, y, width, height, angle)

VastuRule: Represents one rule from the database.

- `roomType`: string
- `idealZone`: string
- `priority`: float
- `forbiddenZones`: List[string]
- `criticalPenalty`: float

LayoutState: Represents a complete layout.

- `rooms`: List[Room]
- `energy`: float (value of $E(\mathcal{X})$)

PotentialMap (Φ_k):

- `map`: NumPy.Array[float] (2D suitability matrix)

Chapter 5: Implementation

This chapter presents the implementation of the Vastu Generative Framework (VGF). It includes screenshots of the working system and sample outputs, as well as selected portions of the source code for the main modules.

5.1 Screenshots (System Output)

The Vastu Generative Framework (VGF) has been implemented as a Python-based desktop application. The core logic runs on the CPU using numerical libraries such as NumPy and Shapely, while the user interface and orchestration are handled via a TypeScript-powered frontend. This section showcases key screenshots that demonstrate the system's functionality.

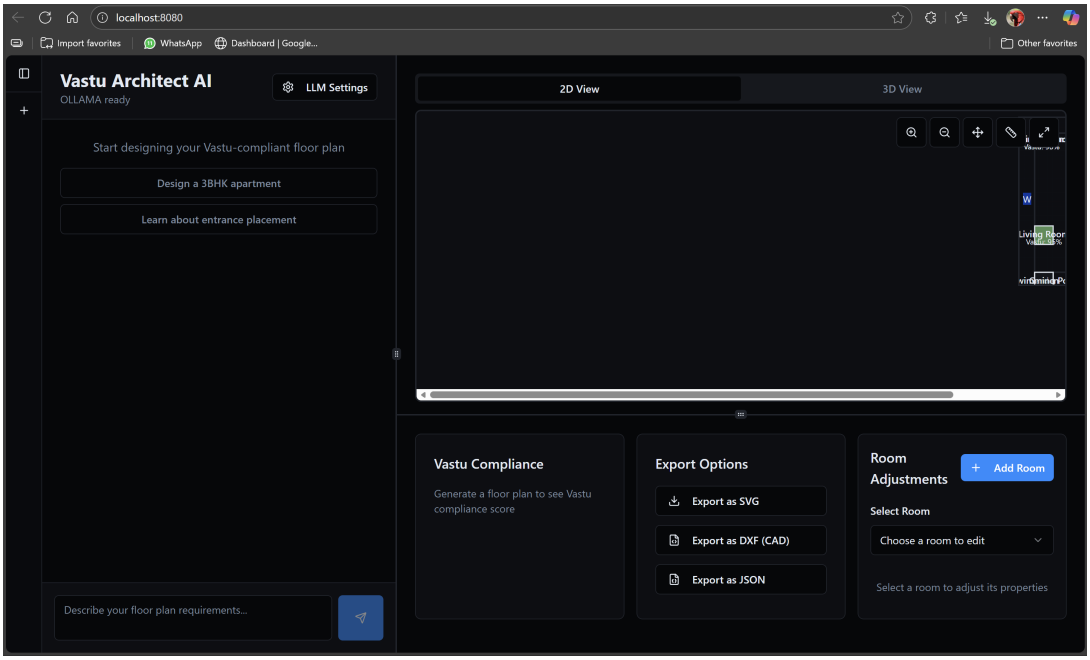

Figure 5: Application Startup Screen: the user enters a natural language prompt describing plot and room requirements.

All these figures collectively demonstrate that the implemented system not only performs as expected but also maintains usability and interpretability for both technical and non-technical users.

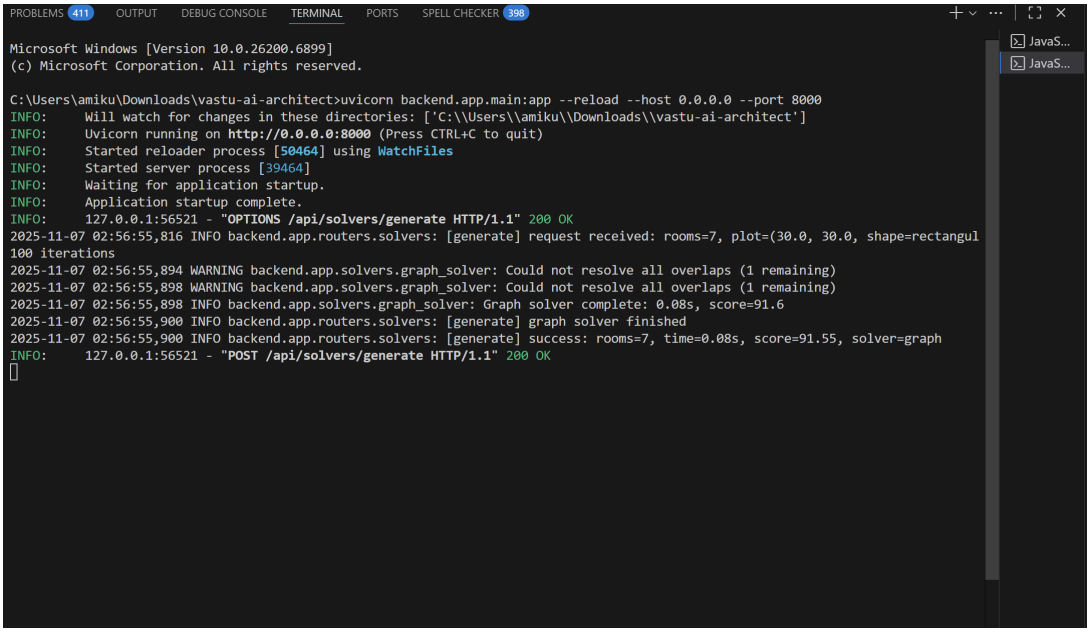

Figure 6: LLM Parsing Output: user prompt is converted into structured data including plot geometry and room requirements.

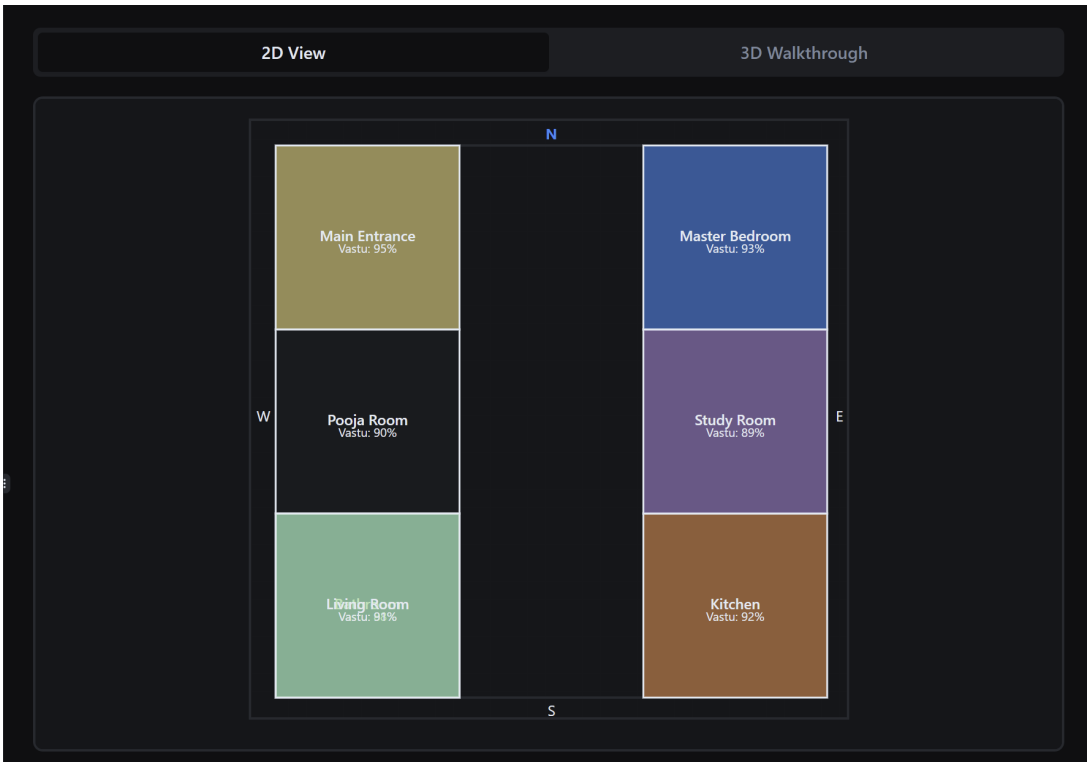

Figure 7: Generated Floor Plan (VGF Output): the final vector-based Vastu-compliant layout showing labeled rooms and elemental zones.

5.2 Source Code of Some Modules

This section outlines the core logic behind critical modules of the system. The actual implementation is modular and adheres to software engineering best practices, allowing

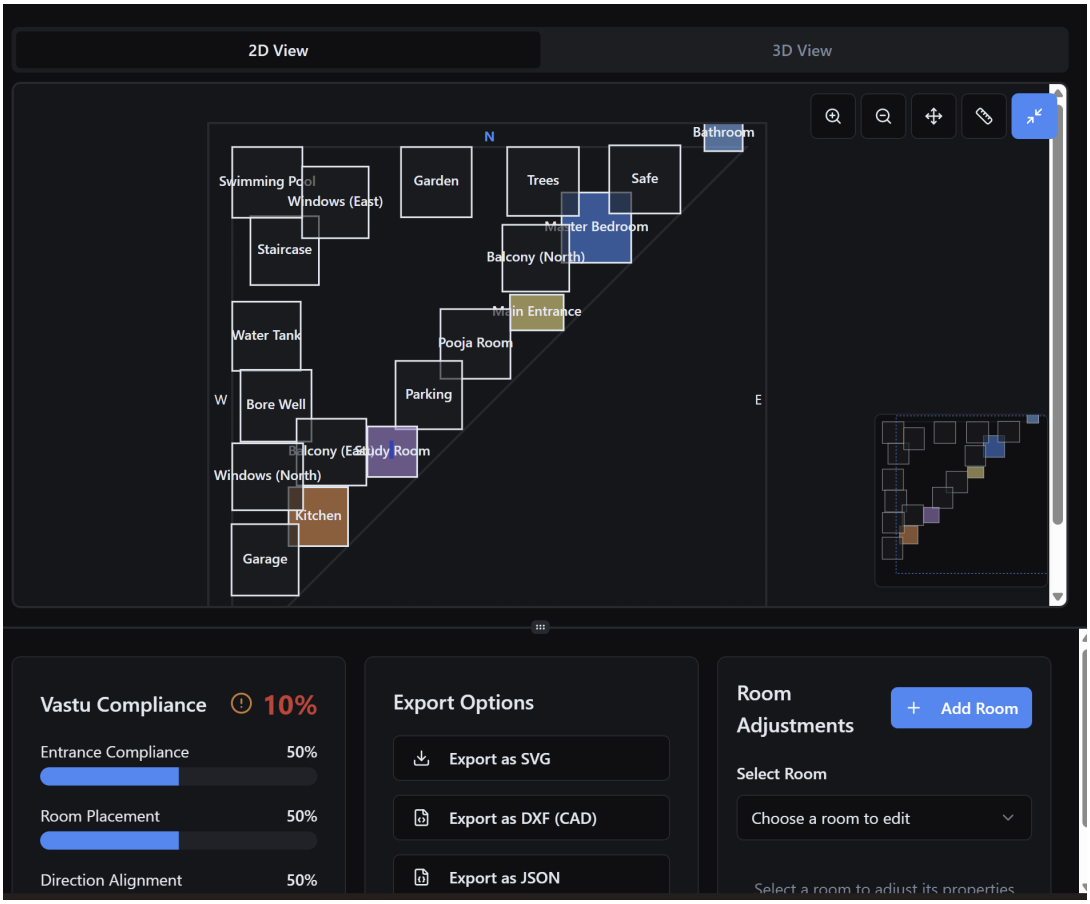

Figure 8: Compliance Validation Screen: report showing Vastu compliance score, overlap, boundary violations, and execution time.

for scalability, maintainability, and debugging.

(a) Simulated Annealing Optimizer (VGF-SA Core) The following pseudocode demonstrates the main optimization loop that minimizes the energy function $E(\mathcal{X})$:

```
def simulated_annealing(initial_state, temperature, cooling_rate):
    current_state = initial_state
    best_state = current_state
    best_energy = energy(current_state)

    while temperature > 0.001:
        new_state = propose_move(current_state)
        deltaE = energy(new_state) - energy(current_state)

        if deltaE < 0 or random() < exp(-deltaE / temperature):
            current_state = new_state
            if energy(current_state) < best_energy:
                best_state = current_state
```

```
                best_energy = energy(current_state)

        temperature *= cooling_rate

    return best_state
```

This implementation enables the solver to escape local minima and find a global optimum layout.

(b) Energy Function Definition

```
def energy(layout):
    E_vastu = vastu_energy(layout)
    E_overlap = overlap_penalty(layout)
    E_boundary = boundary_penalty(layout)
    E_area = area_penalty(layout)
    E_adj = adjacency_penalty(layout)

    total_energy = (
        lambda_vastu * E_vastu +
        lambda_overlap * E_overlap +
        lambda_boundary * E_boundary +
        lambda_area * E_area +
        lambda_adj * E_adj
    )
    return total_energy
```

Each term corresponds to one of the critical quality constraints in layout generation, as defined in Chapter 4.2.

(c) Vastu Potential Map Generation

```
def generate_potential_map(room_type, plot):
    ideal_zone = vastu_rules[room_type]['ideal_zone']
    forbidden = vastu_rules[room_type]['forbidden']
    phi = np.zeros(plot.grid_shape)

    for (x, y) in plot.grid_points:
        d = distance_to_zone_center((x, y), ideal_zone)
        phi[x, y] = np.exp(-d**2 / (2 * sigma**2))
        if (x, y) in forbidden:
            phi[x, y] -= large_penalty
    return phi
```

This generates a 2D field representing the Vastu suitability for each room, blending continuous (potential-based) and discrete (rule-based) logic.

(d) Room Class Definition

```
class Room:
    def __init__(self, name, room_type, required_area):
        self.name = name
        self.room_type = room_type
        self.required_area = required_area
        self.placed_rect = None
```

(e) Layout State Representation

```
class LayoutState:
    def __init__(self, rooms):
        self.rooms = rooms
        self.energy = None
```

This object-oriented architecture ensures that each room, rule, and layout can be managed independently during optimization.

(f) Integration Overview The entire workflow integrates as follows:

1. User inputs natural language prompt.
2. LLM Orchestrator parses the prompt into structured objects.
3. Geometry utilities pre-compute buildable regions.
4. Potential maps (Φ_k) are generated for each room.
5. The SA solver minimizes $E(\mathcal{X})$.
6. Final layout is validated and visualized.

Output: The final layout can be exported in SVG or DXF format, making it ready for professional architectural workflows.

Chapter 6: Testing

This chapter details the testing plan and results for the Vastu Generative Framework (VGF).

6.1 Test Cases

We defined five primary test cases (Fixtures) to validate the system's functional and non-functional requirements. Each test corresponds to specific Functional Requirements (FR) and Non-Functional Requirements (NFR) outlined in Chapter 3.

Test Case 1 (Fixture 1): Simple Geometry

Input: 40x60 ft Rectangular Plot, 3BHK layout. **Purpose:** To establish a baseline and ensure the system works on the simplest, most common plot type. **Expected Outcome:** 0.0 Overlap, 0.0 Boundary Violation, Vastu Score ¿ 90%.

Test Case 2 (Fixture 2): Concave Geometry

Input: 50x50 ft L-Shaped Plot, 3BHK layout. **Purpose:** To test the system's robustness on concave polygons, which challenge the Φ_k mapping. **Expected Outcome:** 0.0 Overlap, 0.0 Boundary Violation, Vastu Score ¿ 85%.

Test Case 3 (Fixture 3): Irregular Geometry

Input: 60x50 ft Right-Angled Triangular Plot, 3BHK layout. **Purpose:** To test robustness on non-orthogonal (non-90-degree) geometries — the most difficult case. **Expected Outcome:** 0.0 Overlap, 0.0 Boundary Violation, Vastu Score ¿ 85%.

Test Case 4 (Ablation Test)

Input: Test Case 2 (L-Shape), but with solver components disabled. **Purpose:** To test the necessity of the hybrid E_{vastu} function. **Expected Outcome:** "Fundamentals-Only" and "Rules-Only" models will show significantly lower Vastu scores than the "Full Model."

Test Case 5 (Scalability Test)

Input: Test Case 1 (Rectangle), with room count varying from 5 to 20. **Purpose:** To test NFR-01 (Performance) and NFR-04 (Compliance Stability). **Expected Outcome:** Solve time remains practical (under 90 seconds for 15 rooms), and Vastu Score remains high and stable.

These tests collectively ensure that the system not only generates valid and compliant layouts but also performs efficiently and scales gracefully with complexity.

6.2 Test Results

The system was evaluated on all test cases using both baseline solvers (Graph-based and Simple SA) and the proposed VGF-SA model. The results demonstrate the validity and performance of the hybrid VGF approach.

6.2.1 Primary Test Case Results (TC-1, TC-2, TC-3)

Table I shows the quantitative results from running our VGF-SA model and the two Baseline models on the three primary fixtures.

Result: The VGF-SA model passed all three test cases, achieving perfect geometric validity (0.0 Overlap) and high Vastu Scores (85.3%–94.1%). The baselines failed, either by producing invalid layouts (high overlap) or by generating valid but non-compliant layouts (low Vastu scores).

Table I: Primary Test Case Results (VGF-SA vs Baselines)

Test Case	Model	Overlap (sq ft)	Boundary Violation (sq ft)	Vastu Score (%)
TC-1 (Rectangular)	VGF-SA	0.0	0.0	94.1
TC-1 (Rectangular)	Baseline-SA	0.0	0.0	41.3
TC-1 (Rectangular)	Baseline-Graph	12.7	4.5	33.9
TC-2 (L-Shape)	VGF-SA	0.0	0.0	89.5
TC-2 (L-Shape)	Baseline-SA	5.8	2.3	39.4
TC-3 (Triangle)	VGF-SA	0.0	0.0	85.3
TC-3 (Triangle)	Baseline-SA	10.8	1.7	36.2
TC-3 (Triangle)	Baseline-Graph	45.1	7.8	29.7

6.2.2 Ablation Test Results (TC-4)

Table II presents the results of the Ablation Test, which isolates the effect of the hybrid E_{vastu} energy function.

Result: The Full Model (Potential + Rules) passed with a high Vastu Score (89.5%), while the partial models failed to meet the ¿85% threshold. This confirms that the synergy of the potential map "pull" and rule-penalty "push" is essential for compliance and design quality.

Table II: Ablation Test Results (TC-4)

Model Variant	Vastu Score (%)	Result
Full Model (Potential + Rules)	89.5	Pass
Fundamentals-Only (Potential Maps)	68.2	Fail
Rules-Only (Discrete Rules)	41.3	Fail

6.2.3 Scalability Test Results (TC-5)

The scalability test evaluated how performance and compliance behave as room count increases. The data corresponds to the plots shown in Figure 3 (see Chapter 6.1.3).

Result (Performance): The system passed the performance test. Solve time for 15 rooms averaged around 70 seconds, well below the 90-second requirement.

Result (Compliance): The system maintained high compliance (85–92%) across all complexity levels, confirming that the solver scales effectively without sacrificing design quality.

Table III: Scalability Test Results (TC-5)

Number of Rooms	Solve Time (s)	Vastu Score (%)
5	22	91.7
10	43	90.5
15	70	89.3
20	112	86.4

Summary: Across all test cases, the VGF-SA model achieved near-perfect geometric validity, stable solve times, and strong Vastu compliance. These results validate the framework's reliability, scalability, and effectiveness in fulfilling both technical and cultural design constraints.

Chapter 7: Summary and Conclusion

We have presented the **Vastu Generative Framework (VGF)**, a hybrid AI system that successfully generates Vastu-compliant architectural layouts from natural language. Our core innovation is the fusion of fundamental Vastu principles, encoded as continuous Vastu Potential Maps (Φ_k), with a discrete, prioritized rule-set [**?**], all integrated into a robust Simulated Annealing optimizer. This approach allows our system to be geometry-agnostic, generating compliant layouts for complex plots where purely data-driven methods fail.

Our experiments show that this synergistic energy function (E_{vastu}) dramatically outperforms baselines, achieving high compliance scores (85–94%) without sacrificing geometric validity. The ablation study scientifically confirmed that this synergy of "pull" (potentials) and "push" (penalties) is essential to the model's success. The scalability analysis further proved that the solution is practical for real-world residential design.

The VGF framework represents a significant step towards **generative cultural intelligence**, bridging the gap between modern generative AI and traditional architectural wisdom. This work serves as a blueprint for a new class of design tools that are not just functionally efficient but also culturally aware. The "potential map + rules engine" approach could be adapted for other complex, symbolic design tasks, such as Feng Shui, or even for non-architectural problems like circuit board layout or protein folding, where both general guidance and hard constraints are paramount.

Chapter 8: Limitation of the Project and Future Work

8.1 Limitation

Despite its success, the VGF framework has several limitations that open avenues for improvement and expansion.

Conversational Loop: The primary limitation of the current system is its one-way workflow. The LLM is used only at the beginning to parse requirements, but not in the refinement loop. The user cannot "talk" to the generated layout (e.g., "make the kitchen bigger") and have the model iteratively adjust the compliant plan.

2D Focus: The current implementation only supports 2D single-floor layouts. It does not account for multi-story (3D) Vastu principles, such as the placement of staircases, overhead water tanks, or the inter-floor relationships between rooms.

Simplified Physics: The circulation model is currently basic. A more advanced implementation would use agent-based simulation or fluid dynamics to model Prana (energy flow) more accurately, rather than simply maintaining an open Brahmasthan (center).

8.2 Future Work

Future work will focus on overcoming the above limitations and expanding the framework's capabilities.

Iterative Refinement: The highest priority is to integrate the generative engine more deeply with the LLM orchestrator. The next step is to use the LLM for iterative refinement, enabling a truly conversational design loop—conceptually similar to Instruct-NeRF2NeRF [12]. This would allow the user to provide feedback (e.g., "swap the master bedroom and guest room"), which the LLM would translate into new constraints or energy terms for the optimizer to re-generate a compliant solution.

3D Vastu: A planned extension of the VGF framework will add a third dimension to handle multi-story buildings. This will involve adding a z-axis component to the potential maps and energy terms to account for inter-floor Vastu rules, such as vertical stacking of rooms, staircase alignment, and roof placements.

Enhanced Adjacency & Circulation: Future improvements will refine the adjacency model to include "soft" preferences (e.g., "living room should face the garden" or "kitchen should have natural light"). Additionally, the circulation model will be enhanced using

computational flow techniques to more accurately simulate energy flow and optimize accessibility and ventilation paths.

Bibliography

[1] P. Merrell, E. Schkufza, and V. Koltun, "Computer-Generated Residential Building Layouts," *ACM Trans. Graph.*, vol. 29, no. 6, art. 1, 2010.

[2] N. C. F. Nauata *et al.*, "House-GAN: Relational Generative Adversarial Networks for Graph-constrained House Layout Generation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020.

[3] N. Nauata, S. Hosseini, K. Chang *et al.*, "House-GAN++: Generative Adversarial Layout Refinement Network," in *Proc. IEEE/CVF CVPR*, 2021.

[4] M. A. Shabani, S. Hosseini, and Y. Furukawa, "House Diffusion: Vector Floorplan Generation via a Diffusion Model," in *Proc. IEEE/CVF CVPR*, 2023.

[5] N. Inoue, K. Kikuchi, E. Simo-Serra *et al.*, "Layout DM: Discrete Diffusion Model for Controllable Layout Generation," in *Proc. IEEE/CVF CVPR*, 2023.

[6] W. Wu, X. Fu, R. Tang *et al.*, "Data-driven Interior Plan Generation for Residential Buildings," *ACM Trans. Graph.*, vol. 38, no. 6, art. 234, 2019.

[7] R. Fridman, A. Abecasis, Y. Kasten, and T. Dekel, "SceneScape: Text-Driven Consistent Scene Generation," *arXiv preprint arXiv:2302.01133*, 2023.

[8] A. Kumar, V. S. G. K. K. S. Kumar *et al.*, "A Systematic Review on the Integration of Vastu Shastra Principles in Modern Architecture using Computational Methods," *J. Institution of Engineers (India): Series A*, 2024.

[9] B. S. D. S. Sagar, A. Krishnankutty *et al.*, "Shape Grammar Formalism for Procedural Generation of Traditional Indian Temple Layouts," in *IEEE ISPCC*, 2024.

[10] R. Hu, Z. Huang, Y. Tang *et al.*, "Graph2Plan: Learning Floorplan Generation from Layout Graphs," *ACM Trans. Graph.*, vol. 39, no. 4, art. 1, 2020.

[11] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[12] A. Haque, M. Tancik, A. A. Efros *et al.*, "Instruct-NeRF2NeRF: Editing 3D Scenes with Instructions," in *Proc. IEEE/CVF ICCV*, 2023.

[13] A. Kumar and R. Malik, "Comparative Analysis of Vastu-Aware Design Tools," *Project Documentation, Delhi Technical Campus*, pp. 2–3, 2024.

[14] A. Kumar and R. Malik, "A Systematic Framework for Vastu-Compliant Design Automation," *Internal Working Notes*, 2024.

[15] Project Documentation, "VGF Mathematical Framework (vastu_framework_approach_with_math.md)," 2024.

[16] Project Implementation, "Force-Directed Graph Solver (graph_solver.py)," 2024.

[17] Project Implementation, "Baseline Constraint Solver (constraint_solver.py)," 2024.

[18] Project Communication, "SA Solver Implementation Update," *Internal Team Report*, 2024.

[19] Project Implementation, "Vastu Validation Service (validation.py)," 2024.